16720 Computer Vision: Homework 2
Scene Classification with a Bag of Visual Words.

Instructor: Martial Hebert
TAs: Varun Ramakrishna and Tomas Simon
Due Date: October 10^{th}, 2011.

Figure 1: **Scene Classification.** Each row depicts a category of scene, such as *building, city, coast, etc.* Our task is to identify a given image as belonging to one of a set of such categories. The rightmost column shows an average image (inspired by the work of Jason Salavon.).

1 Instructions

- **IMPORTANT:** A complete homework submission for this homework consists of three parts. A pdf file with answers to the theory questions and two directories of matlab files for the implementation portions of the homework. You will be

implementing a baseline and a custom implementation. The code for your **baseline** implementation should be placed in a subdirectory named *baseline* and the code for your **custom** implementation should be placed in a subdirectory named *custom* under your main submission directory that is named after your andrew id.

- You should submit your completed homework by placing all your files in the directory named after your andrew id and upload it to your submission directory on Blackboard. The submission location is the same section where you downloaded your assignment. Do not include the handout materials in your submission.
- All questions marked with a **Q** require a submission. For the implementation part of the homework please stick to the function headers described.
- Final reminder: **Start early! Start early! Start early!**. This homework can take considerably more time than the previous homework. You will want to get your baseline implementation coded up as soon as possible so that you can spend time tuning parameters and trying out different methods for your custom impementation.

2 Scene Classification with a Bag of Visual Words

Scene classification is the task of categorizing the environment in a given image as belonging to one of a set of scene classes such *beach*, *mountain*, *city*, etc. In this assignment, your task is to build a complete scene classification system using the *bag of visual words* framework that you learnt in class.

Current research in image classification is often performed on a benchmark dataset with different research groups competing in a **Recognition Challenge**[1]. This assignment will be in the style of such a benchmark challenge, with the top performing submissions earning extra credit. To demonstrate the challenges of the problem we will be working with a subset of the **SUN** dataset[2], which is a large dataset consisting of hundreds of scene categories. We will be using 8 categories from this dataset for this assignment. Scene Classification is still an open area of research, with performance varying across datasets and methods, **so do not expect very high accuracies.** Note that *chance* on this data set would be 12.5 %.

In the course of implementing any real-world computer vision system, you will encounter a series of design choices - which filters to use? which distance function? etc. This assignment has been designed to allow you to make your own design choices at each stage.

You will all build a common baseline **Bag of Visual Words** framework, but will be asked to improve this baseline in a custom implementation: you can choose the specific features you use, how exactly to build the histograms, the particular histogram comparison functions, etc. You are free to re-use the functions you implement in the baseline

[1] Eg. PASCAL VOC challenge: `http://pascallin.ecs.soton.ac.uk/challenges/VOC/`

[2] `http://groups.csail.mit.edu/vision/SUN/`

implementation for your custom implementation. The submission for this homework will be a little different. You will be submitting two directories of matlab files.[3]

The bag of words framework is useful in this setting as the scene is usually well described by its textural properties. Two prominent papers which achieve the same task using a bag of words framework are:

> [1] J. Winn, A. Criminisi and T. Minka, *Object Categorization by Learned Universal Visual Dictionary*,ICCV 2005 http://johnwinn.org/Publications/papers/Winn_Criminisi_Minka_VisualDictionary_ICCV2005.pdf
>
> [2] L. W. Renninger and J. Malik, *When is scene recognition just texture recognition?*, Vision Research 2004.http://www.cs.berkeley.edu/~malik/papers/renningermalik.pdf

For this assignment you will have to read portions of these papers, however you do not have to know or understand every single detail. The relevant sections will be specified.

The following sections describe the construction of a baseline system that you will implement.

2.1 Filter Bank (20 points)

The first step is to create a filter bank, i.e, a series of filters that captures different visual properties. Winn et al[1] implements 4 Gaussians of different sigmas, 4 Laplacian of Gaussians and 4 Derivative of Gaussians. Your baseline implementation should use these filters.[4] Please note that we are leaving the design details of the filters (scale, orientation, support etc) to you. It might require you to try different scales and orientations to get the best performance.

Q1.1 Create and upload the function in your *baseline* directory:

```
[filterBank] = createFilterBank()
```

This function returns `filterBank` which is a cell array. [5]

Q1.2. Visualize the filters you have created in a single picture[6]. Annotate each filter with the parameters used to create it (scale, orientation, type) as in figure 2.

Q1.3 What kind of visual properties are the different type of filters (Gaussian, LoG, DoG) designed to pick up?

[3] Read the instructions.

[4] You can use the **fspecial** function in MATLAB to implement these filters.

[5] A cell array in MATLAB is a structure for storing arrays of different sizes. Check MATLAB documentation for usage information.

[6] Use the matlab command **subplot**

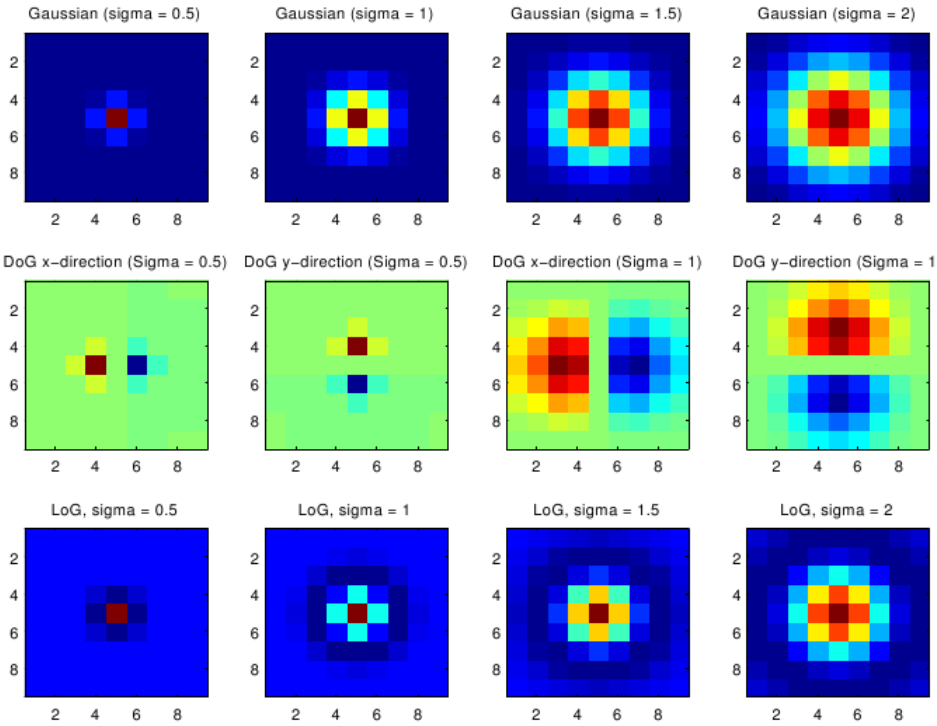


Figure 2: Example filter bank

Q1.3b What filters did you use in your custom implementation? What are they designed to pick up? Submit a plot of the filters you used in your custom implementation as in **Q1.2**.

For your custom implementation, you might want to consider using different features, such as Gabor filters, oriented gaussian derivatives, etc.

2.2 Filter Responses (10 points)

Q1.4 The `extractFilterResponses` function should take in a 3-channel RGB image `I` and apply the filter bank `filterBank` at every pixel using the CIE Lab colorspace[7]. `filterResponses` is an M-by-N matrix, where M is the total number of pixels in the image and N is the number of responses per pixel. As described in Winn et al. [1], the filters are applied separately to each of the CIE Lab channels of the input image. Please use the provided function `[L,a,b] = RGB2Lab(R,G,B)` to convert the RGB image to Lab[8]. In your baseline directory create and upload the function:

[7] The CIE Lab colorspace will be discussed in class soon.
[8] You need to convert the image to `double` before applying the filters.

```
[filterResponses] = extractFilterResponses(I,filterBank)
```

2.3 Vector Quantization (10 points)

We will now create a dictionary of visual words from the filter responses we computed in the previous section. In the limit we could have a distinct *word* for each vector of filter responses from the previous section, however, we will seek a more compact dictionary by performing a type of vector quantization (or clustering) called k-means. After clustering, similar vectors will be represented by the same *visual word*.

Rather than implementing the texton-merging algorithm described in Winn et al. which infers the optimal size of the dictionary, for your baseline system you will be using a library with fixed size (e.g.,, $K = 25$), as described in Sec. 3.1.3 of Renninger and Malik [2], we will perform a simple k-means clustering on the filter responses. ince it is computationally expensive to use all the features from every pixel in the training images, we will extract the filter responses at α random[9] pixels per image (e.g.,. $\alpha = 100$), and then concatenate them into an enormous matrix `GiantFeatureMatrix`.

Q1.5 In your baseline directory, create and upload the script:

`createTextonLibrary.m`

In this script, you will use `extractFilterResponses` over each training image. If you have T training images, then the dimensions of `GiantFeatureMatrix` should be αT-by-N, where again N is the number of filter responses. Our main hope is that these centers capture what it means to be a texture that belongs to that class. To generate the texton library, we will cluster the filter responses using the built-in Matlab command `kmeans`:

```
[res, TextonLibrary] = kmeans(GiantFeatureMatrix, K,
                    'EmptyAction','drop')
```

The `EmptyAction` parameter means that if you input too large a `K`, then redundant cluster centers are dropped. `TextonLibrary` contains the cluster centers that K-means has found in `GiantFeatureMatrix`, e.g.,, this matrix should be K-by-N. [10] Save `TextonLibrary` to file as `TextonLibrary.mat` and also **submit this electronically**.

For generating the texton library, only use the `train` files in `data.mat`. If the clustering is taking over 10 minutes to run, choose a smaller number of clusters and/or fewer sampling points. When initially debugging your code, you may wish to use a smaller set of files specified in `data_debug.mat`. You are free to experiment with setting different sizes for the dictionary or implementing a method to infer the optimal size in your custom implementation.

[9] I used `randperm` to perform the random sampling.

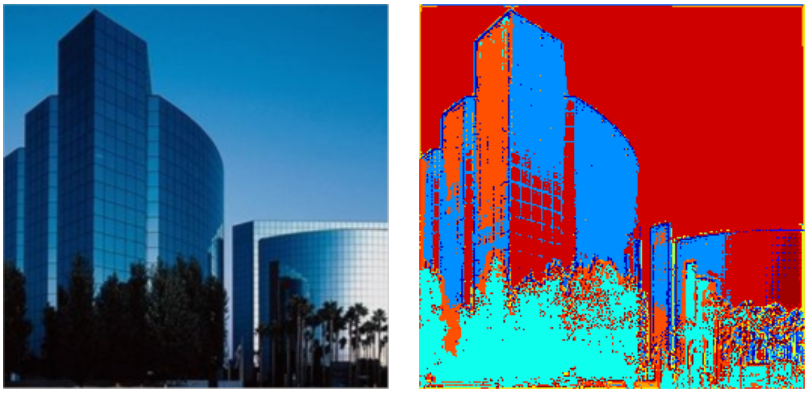
[10] Don't worry if MATLAB throws a did not converge warning, we are only interested in getting a few tight clusters.

2.4 Scene Representation (5 points)

Q1.6 Given an image, filter bank, and a texton library, we need to assign each pixel in the image to its closest texton in the library. Create and upload the function:

```
[textonMap, textonHisto] = textonify(I, filterBank, TextonLibrary)
```

`textonMap` should be a matrix with the same width and height as `I` with each element assigned the respective pixel's closest texton id. The closeness of two vectors is defined by Euclidean distance/L2 norm. To perform all pairs of distance operation efficiently you might want to use the matlab function **pdist2**. An example texton map image is shown in Fig[3] (generated using `imagesc(textonMap)`). `textonHisto` should be a vector of length K that is the histogram of what textons are present in the image; the histogram should be normalized sum to 1. (Hint: `hist`).



(a) (b)

Figure 3: (a) Input image. (b) Resulting texton map.

2.5 Comparing histograms

We have now constructed a histogram of visual words `textonHisto` describing each of our images, but we must still determine how to compare them. As we saw in class, our baseline comparison function will be the so-called *histogram intersection* function, which for every histogram bin takes the minimum value of each, and sums up the result, i.e., $\sum_{i=1}^{K} \min(x_i, y_i)$ or in Matlab, `sum(min(x,y))`. Note that this measure is a *similarity* measure rather than a distance, that is, larger values mean that the two histograms are more alike.

Q1.7 Create and upload the function in your baseline directory:

```
[histInter] = distanceToSet(textonHist, histograms)
```

Where `textonHist` should be an $K \times 1$ histogram with K histogram bins, and `histograms` should be a $K \times T$ matrix containing a set of T histograms (from T training images)

arranged along the columns. This function should compute the histogram intersection similarity `histInter` , of size $1 \times T$ (the similarity from `textonHist` to each element in the training set, in order). Hint: you can use `bsxfun` to avoid loops and duplicating memory. In your custom implementation you may want to try other distance functions, such as the *chi-squared* distance function.

→ Aside: you should now have enough tools to build a basic scene classification system. If you can't wait to see some results, you can skip ahead to Sect. 3, which discusses how to put the pieces together.

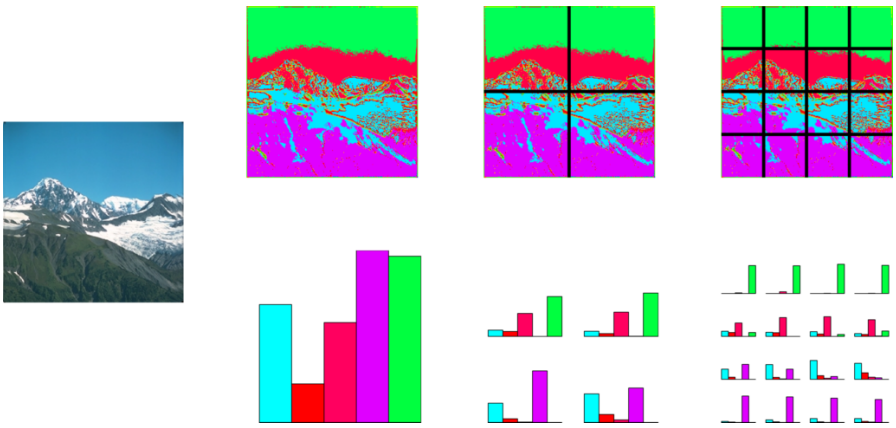


Figure 4: The grid used for spatial pyramid matching. Leftmost, original image. Top row, from left to right, levels 0, 1, and 2 of an $L = 2$ spatial pyramid, with corresponding grid cells. Bottom row, the corresponding histograms for each of the cells.

2.6 Spatial Pyramid Matching

The "bag-of-words" histogram of the previous section discards all spatial information about the location of the features in the image. It therefore does not capture information about the relative arrangement of the features or the shape, and is considered "orderless".

In this section, we will explore an extension called *spatial pyramid matching*, which adds approximate geometric correspondence to the representation. We will follow the approach outlined in the following papers:

> [3] S. Lazebnik, C. Schmid, and J. Ponce, *Beyond Bags of Features: Spatial Pyramid Matching for Recognizing Natural Scene Categories*, CVPR 2006. http://www.di.ens.fr/willow/pdfs/cvpr06b.pdf

For more information on the pyramid matching kernel function, you can also look at:

> [4] K. Grauman and T. Darrell. *The Pyramid Match Kernel: Discriminative Classification with Sets of Image Features*, ICCV 2005. http://www.cs.utexas.edu/~grauman/research/projects/pmk/pmk_projectpage.htm

Be aware that these papers go into some detail concerning the machine learning aspects, and how to build complex classifiers. We are interested only in the representation from the point of view of computer vision, so you don't need to understand every section[11].

The idea is easier to understand visually, as shown in Fig. 4. Intuitively, we are going to divide each image into a grid of cells or *spatial bins*. When comparing two images, we will only match features that fall into the same corresponding cell in each image. This means that for each cell, we will construct a histogram of visual words, and match it to the corresponding cell's histogram in the other image.

There is one further detail: we will do this for different sizes of grids or *pyramid levels*: at each level, the width of the cells will decrease by a factor of 2. When matching, we will always match across corresponding levels of the resulting pyramid, and weight the matches at each level by a certain factor (because matches at a finer level of the pyramid should be more significant than matches at a coarser level, and because we don't want to double count matches).

We will construct a pyramid with $L+1$ levels, and number them $l = \{0, 1, \ldots, L\}$, where level $l = 0$ is the coarsest (largest cell width, equal to the image width). The weight[12] w_l given to each level l will be: $w_0 = \frac{1}{2^L}$, $w_l = \frac{1}{2^{L-l+1}}$ (for example, for $L = 2$ we have $w_0 = \frac{1}{4}$, $w_1 = \frac{1}{4}$, $w_2 = \frac{1}{2}$).

2.6.1 Implementation details

Instead of computing a separate histogram for each pyramid cell at each level, and then computing the weighted sum of the separate histogram intersections, we will implement this operation by concatenating all the histograms from all cells and levels into one long vector. Note that if we multiply each cell's histogram by the weight w_l of the corresponding level and concatenate them all into one vector, the histogram intersection of this long vector will return the same value as if we had computed the histograms separately, and added their weighted similarity (because $c\min(x_i, y_i) = \min(cx_i, cy_i)$).

The basic algorithm to compute a spatial pyramid histogram representation is the following:

1. Starting with the finest level ($l = L$), compute the histogram within each of the spatial bins (there are 4^l of these bins).

2. Compute the histogram for the next coarser level. Note that the histogram of each of the cells in this level will be the sum of histograms of 4 cells from the finer level.

3. Repeat the above step until level $l = 0$, which should be a histogram for the whole image (namely, equal to `textonHisto` from the function in **Q1.6**).

[11] For those of you who are interested, Grauman and Darrell have a version of the paper in the Journal of Machine Learning Research which goes into extensive detail.

[12] See the paper for an explanation of these weights.

Q1.8 Create and upload the function in your baseline directory:

```
[histPyramid, vecHistPyramid] = createSpatialPyramid(textonMap, K, L)
```

This function should create a pyramid with $L + 1$ levels, for histograms of length K. You can assume that the width and height of `textonMap` are evenly divisible by 2^L.

The output `histPyramid` should be a cell array such that `histPyramid{1}` corresponds to the histogram at level $l = 0$ (a vector of size $K \times 1$), `histPyramid{2}` is a vector of size $K \times 4$ (the concatenated histograms of all the cells at level $l = 1$, in left-right, top-bottom order), and so on until `histPyramid{L+1}` . One easy check to perform on your output is that `histPyramid{1}` should be equal to `textonHist`.

The output `vecHistPyramid` of size $H \times 1$ where $H = \frac{K}{3}(4^{L+1} - 1)$ should contain a vectorized and weighted (according to the weights w_l given above) version of this histogram. That is, it will be the concatenation, in the order given above (coarse to fine, left-right, top-bottom) of all the histograms, where all the histograms of level l are multiplied by w_l. It is fine to use for loops in this function.

3 Scene Classification

We will now combine all our previous work into a system which can categorize an image. Classification experiments in general are conceptually split up into two phases: training and testing. The training phase takes in a large set of images for which we know the corresponding label (sometimes called *ground truth*) and usually builds some statistical model of the data. The testing phase is presented with new images, and uses this learnt model to produce a best-guess label for each. Ground truth labels in the testing phase are only used to measure performance.

In order to manipulate the dataset more conveniently, we have provided you with two index structures, named `train` and `test`, in `data.mat`, corresponding to the training and testing images. Each of these structures contains three variables: `files`, `labels`, and `classes`, where `files` contains the image filenames, `labels` contains a string with the corresponding scene label for the images (e.g., "coast", "city"), and `classes` is a vector of length T with a number that identifies which of the scene categories the image belongs to. For example, `train.files{1}` could be `'building01.jpg'`, `train.labels{1}` would be `'building'`, and `train.classes(1)` would be 1. The categories, in order, numbered 1-8, are available in a variable `categories`, also in `data.mat`:

```
categories = { 'building', 'city', 'coast', ...
'country', 'forest', 'highway', 'mountain', 'street' };
```

To test and debug your code, you may choose to use the smaller set of images specified by `train` and `test` in the file `debug.mat`. Do not worry about the performance on the debugging set. You will need to experiment with different filters, texton construction (α, K), and the levels of the spatial pyramid (L), to get good results.

3.1 Experiment Setup

Q1.9 (5 points) Create and upload the script in your baseline directory:

`prepareTrain.m`

that uses the `TextonLibrary` from Sect. 2.3 to perform the necessary steps to compute a `vecHistPyramid` for each of the T training images. Store each pyramid as a column into a matrix called `TrainHistograms`. That is, `TrainHistograms` is an $H \times T$ matrix of histogram pyramids. These should be in the same order as specified in `train.files`, etc. So `TrainHistograms(:,i)` should be the histogram for file `train.files{i}` of class `train.classes(i)`. Save and upload `TrainHistograms.mat`.

Also create and upload another script (which is almost identical)

`prepareTest.m`

that performs the same operations over the *testing* images (using the same `TextonLibrary`). Save and upload `TestHistograms.mat`.

3.2 Testing

As our baseline implementation, we will use a simple 1-nearest-neighbor (1-NN) classifier to categorize new image scenes. In 1-NN, the histogram of each testing image in `TestHistograms` is compared with all histograms of `TrainHistograms` under some distance metric. The training image with the highest similarity (or lowest distance) is selected as the nearest neighbor and its category label is used as the prediction for the test image. As a baseline, we will use the histogram intersection similarity you implemented above. (For your custom implementation, you are free to use any classification scheme you want. One example of a simple extension is using k-nearest neighbors.)

Q1.10 (5 points) Create and upload the script in your baseline directory:

`sceneCategorize.m`

that iterates through each testing image and performs 1-nearest-neighbor (1-NN) scene categorization. This script should create a vector `EstimatedTestClasses` of length `size(TestHistograms,2)`, where each entry `EstimatedTestClasses(i)` contains the buest-guess class identifier for the corresponding testing image.

3.3 Evaluating performance

There are several different measures to evaluate the performance of classifiers. We will use some of the simplest and most common, *accuracy* $= \frac{\text{\# of correctly classified test images}}{\text{\# of total test images}}$, and the *confusion matrix* summarizing the performance[13].

[13] You can compute the accuracy from the diagonal of this matrix, and many other performance measures with similar operations.

Construct an 8-by-8 confusion matrix where entry $[i, j]$ are the number of instances from class i that were predicted as class j. Table 1 shows an example of such a confusion matrix on this dataset with an accuracy of 0.6484 (or 64.84%).

- *Performances will vary due to different design decisions.*
- *This is an ongoing research topic. Even on this reduced set, don't expect to get* 100% *accuracy.*

	Building	City	Coast	Country	Forest	Highway	Mountain	Street
Building (1)	9	2	2	1	0	0	2	0
City (2)	2	8	1	1	0	1	1	2
Coast (3)	2	1	7	0	0	2	2	2
Country (4)	0	1	1	9	2	1	2	0
Forest (5)	1	0	0	0	14	1	0	0
Highway (6)	0	0	1	2	0	13	0	0
Mountain (7)	0	0	2	0	2	1	10	1
Street (8)	0	2	0	0	0	0	1	13

Table 1: Confusion Matrix

Q1.11: Create and upload a *function* in your baseline directory:

```
function [Accuracy, TestConfusion, EstimatedTestClasses] =
                runTrainTest( train, test )
```

Which computes accuracy, the confusion matrix, and returns the estimated class labels as defined above. This function should take in the training and testing structures specified above, and run all the necessary steps (by calling the scripts) to run the full system from top to bottom. `Accuracy` should be a scalar, `TestConfusion` an 8 by 8 matrix as above, and `EstimatedTestClasses` as defined above. Save these three variables and upload them as a file `TestResults.mat`. Please create the same script for your custom implementation.

Q1.12: In your *answer sheet*: Write down your confusion matrix and your accuracy. What can you conclude from your confusion matrix? Give one or two example images showing why certain classes are confused more than others. What were your parameters and how did you choose them?

4 Extra Credit: Custom Implementation

In the previous sections you have built a baseline bag-of-words scene classification system. You now have the basic tools and workflow for designing your own scene classifcation

system. Experiment with different filters, distance functions, clustering methods and classifiers to improve your performance. Submit all your files for your custom implementation in the **custom** directory with a script that will generate your results and display the confusion matrix. Also include a write-up detailing the design decisions you made and the parameters you used in your implementation. The custom implementation will be graded for extra-credit in the following way:

- The top 10 performing submissions will receive 30 extra credit points
- The next 10 will receive 20 extra credit points.

Implementation and Debugging Hints

- Start early! This assignment will take significantly longer than the previous homework.
- Sometimes it is hard to know if things are working as they are expected to. Make sure that you are getting meaningful results early on. Try to visualize all partial results before continuing to the next step.
- It is often useful to convert your data from class uint8 to double, otherwise your calculations may be truncated and rounded, drastically throwing off results.
- There are efficient, inefficient and very inefficient ways to implement functions in Matlab. Each image should only take a few seconds to process. If your entire program is taking over an hour to run, talk to a TA about how to write efficient Matlab code.
- Refrain from reading all the training images and testing images into memory at once. This will cause a surge in memory usage, and slows down the program substantially. Read in each image sequentially